Lab 7 Fuzz Testing

Jamula Soham Rao | PES2UG24CS202 | Section D

Assignment 15 Movie Tickets | Hypothesis and pytest

1. Original application failure

`python movie_tickets.py`

Pressing Enter without entering a seat makes the original application stop with IndexError. It never reaches the price and rating prompts.

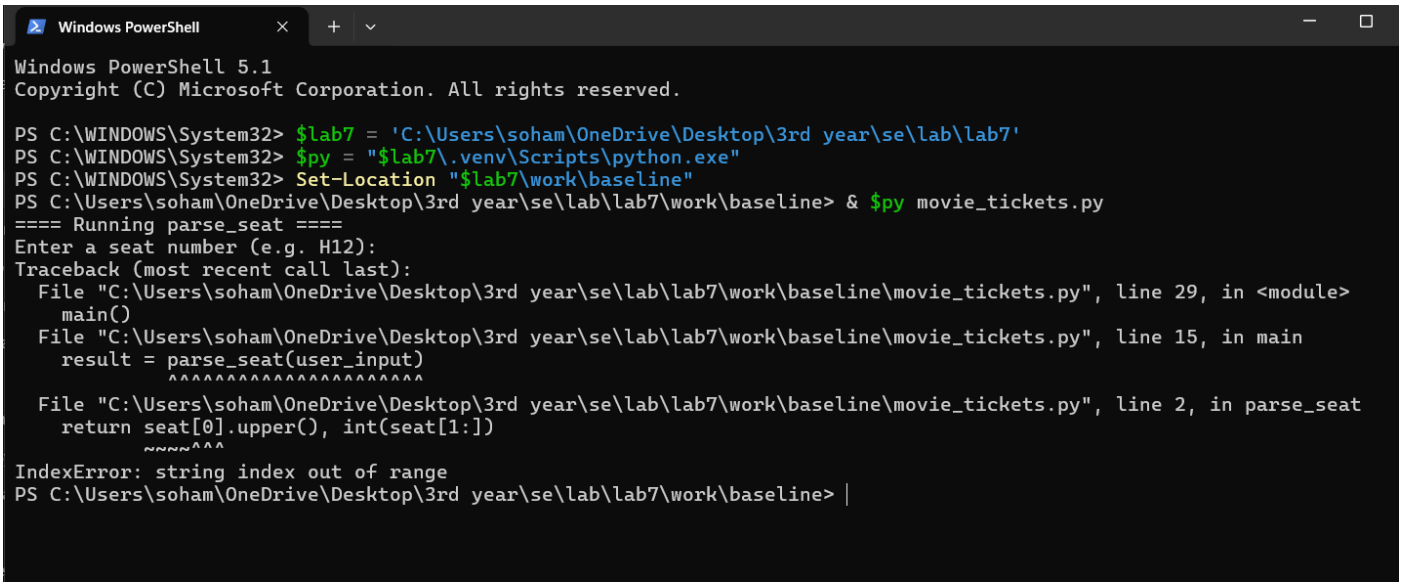


Figure 1. Student-captured PowerShell screenshot of the original application crashing on an empty seat.

The screenshot shows the command, the empty seat prompt, and the traceback. Separate recorded runs retain stdout/stderr, input, exit status and source hashes in the evidence directory.

2. Failing Hypothesis tests

`python -m pytest -q --tb=line -k no_crash test_movie_tickets.py`

The completed tests were first run against the original source. All three no-crash properties failed; 59 other cases were deselected for this focused baseline run.

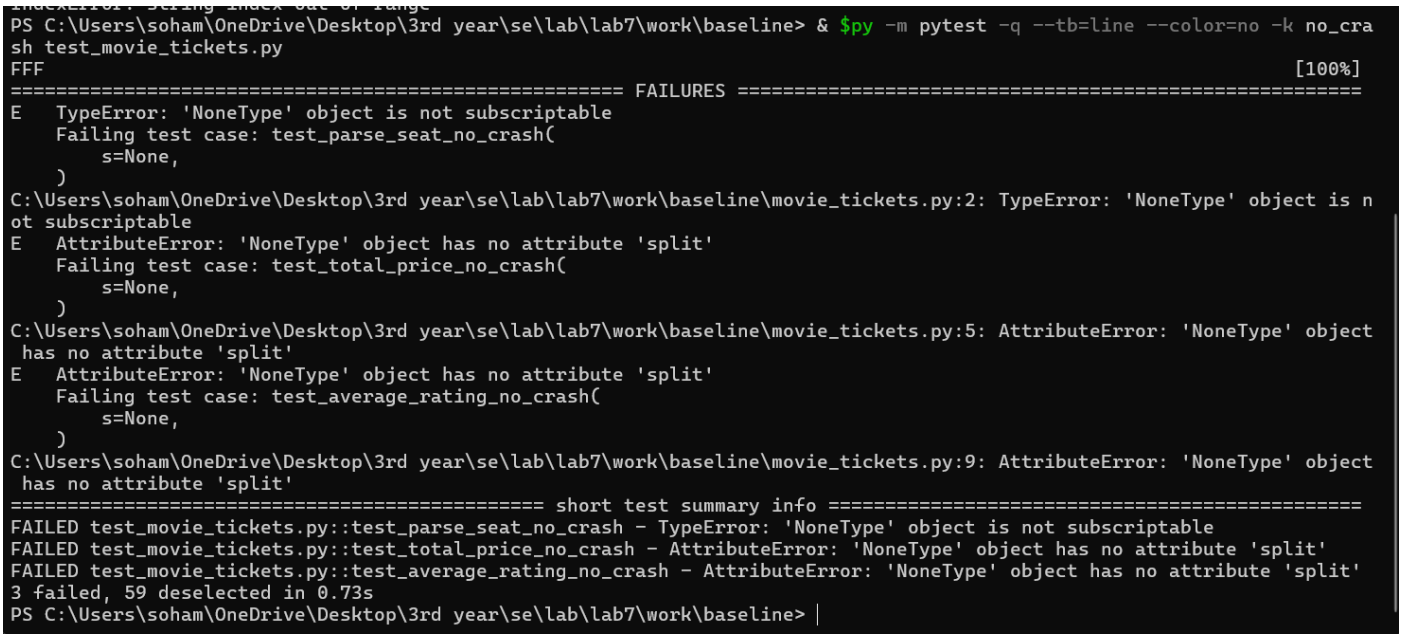


Figure 2. Student-captured PowerShell screenshot showing three failed properties, their minimized inputs, and 59 deselected tests.

Hypothesis reduced the failing input to None for all three functions. parse_seat raised TypeError; total_price and average_rating raised AttributeError when calling split on None.

Separate recorded probes also confirm IndexError for an empty seat, ValueError for 150,,250, ZeroDivisionError for empty ratings, and OverflowError for a 400-digit price. These are additional probes, distinct from the three minimized fuzz examples.

3. Fixed application output

`python movie_tickets.py`

The fixed application receives an empty seat, malformed prices 150,,250, and empty ratings. It displays a validation message for each prompt, reaches all three functions, and returns to PowerShell without a traceback.

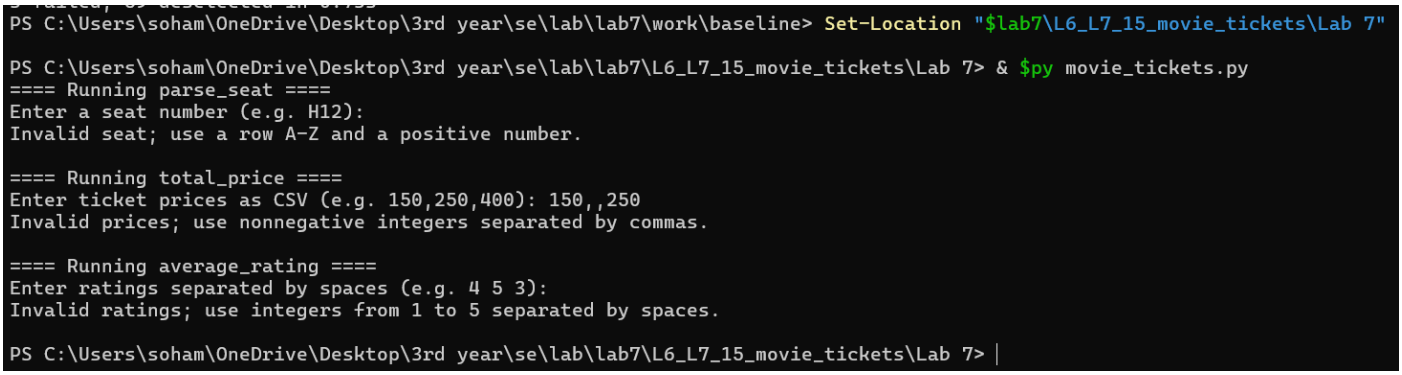


Figure 3. Student-captured PowerShell screenshot of all three invalid inputs handled by the fixed application.

The functions validate types, length, format and domain before computation. Invalid input returns None; the CLI explains the error and continues. A malformed CSV rejects the whole input rather than silently discarding a bad price.

A separate valid-input run confirms the original examples still work: H12 produces ('H', 12); 150,250,400 produces 944.0 with GST; 4 5 3 produces an average of 4.0.

4. Passing fuzz tests after fixes

```
python -m pytest -q test_movie_tickets.py
```

All 62 tests pass. The accompanying statistics run records 2700 passing generated examples across nine Hypothesis properties, plus 53 explicit and parameterized regression cases.

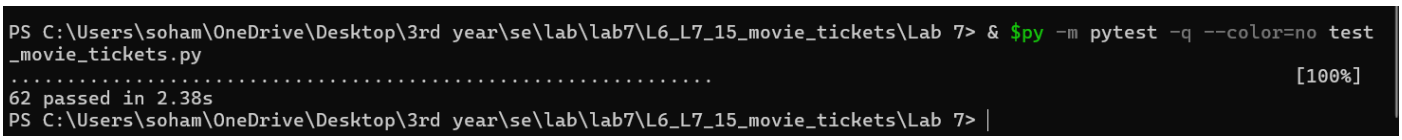


Figure 4. Student-captured PowerShell screenshot showing 62 tests passed in 2.38 seconds. The 100% indicator is pytest progress.

Validation policy adopted for this solution: strings are limited to 4096 characters; seats use one ASCII row letter and a positive number; prices are nonnegative ASCII integers; ratings are ASCII integers from 1 to 5. Surrounding whitespace is accepted. Invalid data returns None, and monetary totals must be finite.

The handout does not supply a full input/error contract, so these domain rules are stated as solution choices. Valid-output properties and an independent Decimal price oracle check correctness as well as crash resistance. This finite fuzz campaign does not establish correctness for every possible input.